

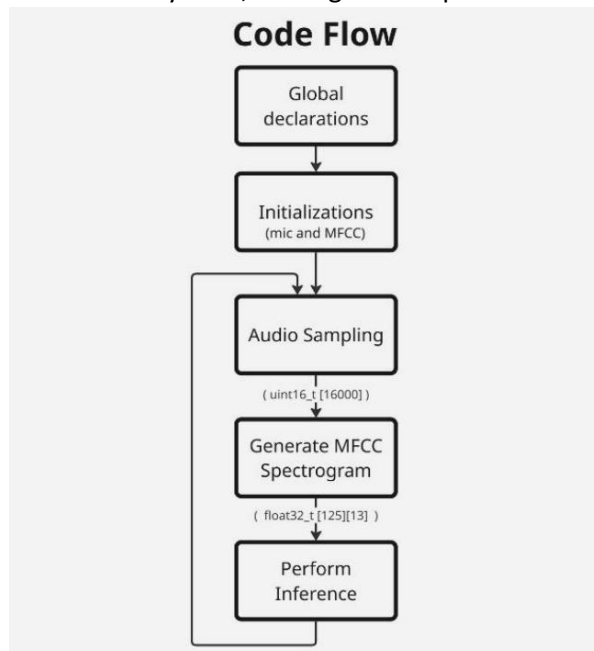
Documentation:

Speech Keyword Detection Application

Lakshmeesh K P

Brief introduction:

- The goal was to explore Zephyr RTOS and TFLM through a speech keyword detection application.
- The model was trained referring Tensorflow's "Simple audio recognition" example available online. [\[1\]](#). It demonstrates pre-processing and training a keyword classification ML model. It trains on 8 keywords. The audio is undergone STFT transformation to obtain features and then trained on a 2D-CNN model.
- The final project consists of a 1D-CNN model trained on MFCC features of these keyword audio files, to improve the accuracy while also compressing features size to make it more compatible for low memory applications. It was deployed on a Raspberry Pi Pico, after fully INT8 quantizing the model, running on TFLM (tensorflow for microcontrollers). It used CMSIS-DSP for audio preprocessing since it provided direct APIs for generating MFCC features.
- The current application takes a 1 sec audio input after every loop. But it can also be extended to be made real-time, such that it always hears for a keyword, making it more practical for real life use cases.



- The problems faced during the project and solutions are also mentioned in the documentation

1. Microphone interfacing

- Hardware setup:
 - o MAX9814 microphone module with adjustable automatic gain control (AGC) was selected to get reliable and clean audio.
 - o It was tested initially using a simple micro-python script, which captures the audio waveform and streams it to PC, where we listen it by converting it to a .wav file.
- Zephyr implementation:
(refer to the [portability basics pdf](#) for detailed info.)
 - o Setting up the ADC:
(references: samples/drivers/adc/adc_dt)
 - We create an overlay file specifying the ADC channel and its configuration and access the ADC node within the application code using ADC specific zephyr APIS

2. Audio pre-processing

- Deviating from the TF example's STFT preprocessor:
In this project, I have used MFCC instead of STFT, for reasons explained below:
 - o For implementing STFT, 'kissfft' library was available as a module within zephyr. However, due to my inexperience with build systems, I was having a hard time setting it up.
 - o Another DSP library was available within zephyr called "CMSIS-DSP". A bonus was that it also included MFCC.
 - Pros:
 - the library was speed optimised for Arm cortex-M/A
 - simpler implementation to the application
 - MFCC spectrogram is much more compact than STFT due to DCT compression (thus saving space); it also tries to mimic human hearing characteristics, thus focusing on the more important parts of the signal. It is also more immune to noise.
 - Having a compact spectrogram (125x13 compared to 124x129) means that we can cut down on the model size by using 1D-CNN instead of 2D-CNN.
 - Cons:
 - non-portable to non-arm MCUs
 - CMSIS-dsp relies on SIMD, DSP, FPU modules for speedup. However, the Pico doesn't have any of those, and doesn't gain any massive speedups.
- Background knowledge on working of STFT and MFCC:
 - o STFT spectrogram (Short-Time Fourier Transform):
 - If we just take the FFT of an audio, we only get the frequency information, but lose out on the information of when they occurred. Thus, the audio is split into overlapping time frames (so as to not miss any audio transitions),
 - a windowing function (like hamming) is applied to prevent sharp discontinuities and FFT is taken.
 - These are stacked together to create a 2D spectrogram. The resulting dimension is a design choice. The tensorflow example has used "square"(124x129) dimensions and correspondingly used a 2D CNN based model.
 - o MFCC spectrogram (Mel-Frequency Cepstral Coefficients):
 - MFCC is an extension of STFT, it also splits the audio into short overlapping time-frames, windowing and calculating FFT of each frame.
 - Then the Mel filter bank is applied on these FFTs, converting the linear frequency scale to a nonlinear one, making it more aligned with human perception. This Mel-scale is linear for

lower frequencies (below 1KHz) and then logarithmic at higher frequencies, to model how human ears are more sensitive to changes in lower pitch than higher. The output contains the energy captured within each filter bank.

- Logarithm of the filter bank output is taken to model humans logarithmic perception of loudness.
- Then, a DCT is applied on the log-mel spectrum to obtain the most significant features from each frame, compressing the features.
- This spectrogram is then used for training the model.
- The dimensions of the spectrogram generated by me was designed to be (125,13). Thus, I decided that a 1D CNN model would be sufficient.

- Using KissFFT library:

- I created a custom STFT api was created using kissfft. (refer my github [\[2\]](#))
(to try testing the original TF example when I was facing memory related issues from not properly aligning the C-array model in the memory, and suspected the issue to be from CMSIS side).

- Library integration:

```
zephyr_include_directories(  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft  
)  
  
file(GLOB app_sources src/*)  
target_sources(app PRIVATE  
    ${app_sources}  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft/kiss_fft.c  
    ${ZEPHYR_BASE}/../optional/modules/lib/tflite-micro/third_party_static/kissfft/tools/kiss_fftr.c  
)
```

- Setting CONFIG_TENSORFLOW_LITE_MICRO=y in prj.conf by default pulls in the files for kissfft. However, if we are using it for any non-ML application, the above directory and sources would need to be included.

- Usage:

- Kiss_fftr_alloc() allocates and initialises the configuration struct for performing fft. We pass in length of real-valued input, choose if we want to do forward or inverse FFT, and pointer to the configuration buffer and pointer storing the length of this buffer. This can be used to get the size of buffer used.
- Then call kiss_fftr() with the set configuration, input and output buffers to get the result.
- You may refer to the above linked project.
We pass in the various configuration settings into stft_init(), and stft() returns the spectrogram after framing, windowing and performing fft on each frame.
stft_free() is used to free the allocated buffers.

Note: It assumes that input is 1sec long at "frames" calculation step, and assumes datatype int16 with range [-32768, 32767] at the normalisation step

- Memory bug:

- kiss_fftr_alloc(256, 0, NULL, NULL) returns a dynamically allocated pointer, which wasn't being malloced properly and failing silently.
- To overcome it, I predefined the required buffers:
kiss_fftr_alloc(256, 0, fft_cfg_buffer, &fft_cfg_buffer_len)
- The correct value of FFT_CFG_BUFFER_LEN was determined by first printing the size returned by kiss_fftr_alloc during a test run and using that as the pre-defined constant.

- Using CMSIS-DSP MFCC library:
 - For MFCC, we require constants like Mel filters, window coefficients, DCT coefficients. This can be generated using *GenMFCCDataForCPP.py* script (modules/lib/cmsis-dsp/Scripts).
 - Generating MFCC Constants:
 - The script relies on SciPy, and in the newer versions required a small modification in *mfccdata.py*: (or see applications/cmsis_gen)
from scipy.signal.windows import hamming, hann
 After this, replace the “hamming” accordingly to make it work
 - A custom .yaml file has to be made to specify the sampling rate, FFT length, no. of mel filters, DCT outputs, fmin, and fmax ($\leq fs/2$ due to Nyquist limit) (Refer to example config files provided by CMSIS)
 - Then run the script using:
python GenMFCCDataForCPP.py -n “consts” -d out_c -i out_h mfccconfig.yaml
 Where “consts” is output file name, out_c/out_h is the destination of .c/.h files, and “mfccconfig.yaml” is the custom config file. The generated files can directly be included in the project.
 - MFCC setup in code:
 - CMSIS provides MFCC instance structures (eg, *arm_mfcc_instance_f32*) in “cmsisdsp/include/dsp/transform_functions.h”.
 - We initialise the MFCC instance (eg: *arm_mfcc_init_f32*) with the required arrays.
 - MFCC transform can then performed (eg: “*arm_mfcc_f32()*”)

```

size used: 2836
cfg address: 0x200017c8
out address: 0x200006d0
FFT Output:
index 0: Real: 32.596748, Imag: 0.000000
index 1: Real: 38.815857, Imag: -0.061893
index 2: Real: 90.735458, Imag: -0.289242
index 3: Real: -73.884689, Imag: 0.353082
index 4: Real: -20.883911, Imag: 0.132956
index 5: Real: -10.869841, Imag: 0.086409
FFT Output:
index 0: Real: 0.003599, Imag: 0.000000
index 1: Real: 0.003737, Imag: 0.086650
index 2: Real: -0.404432, Imag: -128.031555
index 3: Real: 0.002855, Imag: -0.155571
index 4: Real: 0.003047, Imag: -0.086439
index 5: Real: 0.003106, Imag: -0.061723
index 6: Real: 0.003134, Imag: -0.048582
index 7: Real: 0.003148, Imag: -0.040301

```

3. TensorFlow for Microcontroller (TFLM) for ML

- Motivation:

- Typical ML frameworks (Pytorch, tensorflow, onnx) are designed to be ran on high performance devices.
- However, Pico has very limited memory, and also no FPU for fast float computations.
- TFLM is one of the few solutions available for this challenge. It is developed by Google to deploy ML applications on edge devices, with a very small footprint.
- Moreover, Zephyr exposes TFLM as a module, which makes it easier for us to include it in our project.

- TFLM basics:

○ MicroMutableOpResolver:

- In standard TF, usually the runtime engine loads all the operations in the memory. However, it is unpractical for microcontrollers to have this overhead.
- In TFLM we have to manually specify the required operations into the MicroMutableOpResolver tflite struct.
- The below snippet can be used after training and quantisation to find out which ops are required:

```
interpreter = tf.lite.Interpreter(model_path="model.tflite")
for op_details in interpreter._get_ops_details():
    print(op_details['op_name'])
```

○ Model C array:

- The entire model is converted to a serialised byte array (flatbuffers), and is read from the ROM during runtime.
- FlatBuffers use a memory-mapped data layout and are accessed directly from memory via pointer offsets, rather than being copied into aligned structures like typical C arrays. Thus, its important to ensure that this file is memory aligned properly.
- To convert a .tflite to a C-array file, we can use a command-line utility “xxd”.

or we can also use a simple python script:

```
def tflite_to_c_array(tflite_path, array_name="model_data"):
    with open(tflite_path, 'rb') as f:
        tflite_content = f.read()
    header_path = tflite_path.replace('.tflite', '.h')
    with open(header_path, 'w') as f:
        f.write(f'#ifndef {array_name.upper()}_H\n')
        f.write(f'#define {array_name.upper()}_H\n')
        f.write(f'const unsigned int {array_name}_len = {len(tflite_content)};\n')
        f.write(f'const unsigned char {array_name}[] __attribute__((aligned(8))) = {{\n ')
        # Format bytes into hex with 12 values per line
        for i, byte in enumerate(tflite_content):
            f.write(f'0x{byte:02x} ')
            if i < len(tflite_content) - 1:
                f.write(' ')
            if (i + 1) % 12 == 0:
                f.write('\n ')
        f.write('\n};\n\n')
        f.write(f'#endif // {array_name.upper()}_H\n')
    print(f"header created at: {header_path}")
if __name__ == "__main__":
    tflite_to_c_array(r'c:\Users\laksh\zephyrproject\zephyr\applications\Speech_enhancement\')
```

○ Tensor arena:

- On PCs, memory space is typically allocated dynamically, but this is risky on MCUs in case the allocation fails during execution.
- TFLM uses tensor arena, which is a contiguous block of memory. All the calculations related to inference happens within this memory block.
- Once verified during initialisation, it is guaranteed to run without running out of memory. We can minimise it later by using “interpreter->arena_used_bytes()” which returns the memory space actually allocated.

○ Quantisation:

- Quantization converts the weights, biases, and activation parameters from higher-precision float32 down to lower-precision int8, while maintaining comparable accuracy.
- This gives a very good 4x reduction in model size. Moreover, allowing MCUs like Pico which don't have an FPU to not have to do more work for float operations.

- There are 3 types of quantisation: Full int8, float16, and dynamic range.
 - Full INT8:
 - Both weights and activations are converted to int8.
 - The following mapping is done:

$$f32_value = Scale \times (int8 - Zero_point)$$
 - Calibration of scale and zero-point constants requires a representative dataset to be passed through the model. The values are analysed to return the optimal constants.
 - Float16:
 - weights and activated are converted to f16. Useful when we require minimal loss in model accuracy.
 - Dynamic range:
 - The weights are converted to int8 precision, while the activations/output are still float32. We don't require a representative dataset to perform this quantisation.
 - TFLM doesn't support dynamic range quantisation.

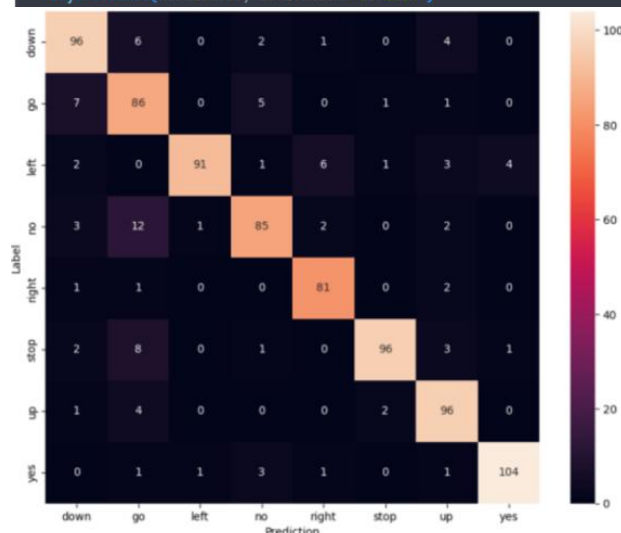
- Model training:

- Tensorflow's "Simple audio recognition" example was referred for training [\[1\]](#).
See github page for [codes](#).

- Final specs:

- MFCC pre-processing was used with a 3 layered 1-D CNN model.
- validation accuracy = 88.3

```
#improved version using avg pooling instead of flattening + additional conv layer
model = models.Sequential([
    layers.Conv1D(16, 4, activation='relu', input_shape=(125, 13)),
    layers.MaxPooling1D(2),
    layers.Conv1D(32, 4, activation='relu'),
    layers.MaxPooling1D(2),
    layers.Conv1D(64, 4, activation='relu'),
    layers.MaxPooling1D(2),
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(num_labels, activation='softmax')
])
```



- I tried checking if increasing no. of DCT coefficients in MFCC would give any improvements, but it was minimal, thus, finally proceeded with 12 coefficients:

# DCT coefficients	Validation accuracy (over 5 runs)	#params	Spectrogram space
13	~0.788	15.86KB	52KB
16	~0.786	15.9KB	64KB
20	~0.776	16.0KB	80KB
26	~0.792	16.17KB	104KB
40	~0.792	16.5KB	160KB

4. Results

- After quantisation, application occupied only ~22% of pico's RAM
- Performed well for 'go', 'no', 'right', 'up', and were well differentiated. However, it was hard to detect 'down', 'stop', 'left' 'yes'.

keyword	Inference confidence
No	~0.757
Go	~0.777
Right	~0.789
up	~0.758

For eg:

'yes' got detected as 'no' frequently. The model was also only detecting it by playing the training dataset audio into the mic, suggesting possible overfittings. This could possibly be because of using the Global average pooling layer instead of the original flatten layer (done for space optimization), which might have removed the sparse information required to distinguish 'yes' and 'no'.

5. Real-time feasibility

- At the default pico configuration (ie 133MHZ clock rate), below are the time taken for each task:

Task 1	Audio sampling	1 sec
Task 2	Spectrogram generation	~1.36 sec
Task 3	TFLM inference	~0.1 sec

For real-time, we need to complete tasks 2,3 during the sampling duration. That was not feasible with the original clock-rate. This issue can be overcome by increasing the clock rate to 255MHz (halving spectrogram generation time) manually. See this [github page](#) for overclocking guidance.

- Further, instead of directly calling zephyr ADC read apis, we have to use DMA for reliable and faster sampling, also enabling the sampling to run in background while task-2,3 can be executed by the CPU in parallel for real-time implementation.
- The core principle is that we perform inference on overlapped 1 sec audio windows, and try to detect keywords within any 1 sec frame with good confidence.

